

增強課程 8：期末綜合實作——真實客戶需求案例

Lecture

en04 ~ en07 每一題都是「生產者視角」的深度練習：自己動手建立 Implicit Enhancement Point (en04)、自己建 Enhancement Spot / BAdI Definition (en05)、自己寫 Implementation (en06)、自己在 Z 程式裡宣告 Explicit Enhancement Point/Section (en07)。這些練習把每種技術的建立機制、排錯眉角都摸透了，但有一個關鍵盲點——一般 **ABAPer** 在真實專案裡，幾乎不會有機會「自己宣告」一個 **Explicit Enhancement Point**。原因很簡單：Z 程式是自己的，想改邏輯直接改程式碼就好，不需要刻意留一個「以後可能有人要插入程式碼」的位置；會主動宣告插入點的，通常是 SAP 自己（在標準物件裡）或套件開發商（賣框架給別人客製）。

真正日常會遇到的工作模式，反而是**消費者視角**：業務單位提出一個需求，需求剛好命中一個**標準物件**（不能碰它的原始碼），這時候要做的是——

- 1. 到標準物件裡找有哪些既有掛勾點（Classic User-Exit / Classic BAdI / New BAdI / Implicit Enhancement / Explicit Enhancement）
- 2. 判斷該用哪一種（en01 已經練過這個判斷力，這題要在真實案例上再練一次）
- 3. 在那個掛勾點上**Implement**客製化邏輯
- 4. 驗證沒有影響到標準流程本身

en08 就是照這個消費者視角，設計三個各自獨立的真實客戶需求，分別對應三種技術：

#	技術	案例	狀態
1	Classic User-Exit	PPCO0001：工單存檔時把 Component Change 存入 Log Table	待建置
2	Implicit Enhancement	CO02/CO03 Cost Analysis 權限漏洞：標準功能不卡使用者權限，補上廠別層級授權檢查	✅ 已完整驗證
3	Explicit Enhancement (消費者視角)	MB52 加 MRP Controller 選取欄位，掛在 SAP 標準程式已經存在的 Explicit Enhancement Point 上	待建置

案例 3 特別值得注意：跟 en07 剛好相反——en07 是「生產者視角」，自己在 Z 程式裡宣告插入點；這裡是**消費者視角**，SAP 標準程式（**RM07MLBS**，MB52 底層報表）裡已經存在別人（另一個開發者）留下的 **ENHANCEMENT-POINT**，這次要做的是在同一個插入點上再掛一個新的 **Implementation**，不用自己宣告任何東西。

案例二：CO02/CO03 Cost Analysis 權限漏洞 (Implicit Enhancement)

需求

使用者在 **CO02**（變更生產訂單）功能表裡做「Cost Analysis（成本分析）」，標準功能**不會檢查使用者權限**——任何登入的使用者都能看到任何工廠的成本資料，需要補上一個**依廠別（Plant）判斷的權限檢查**，沒有權限就跳出錯誤訊息擋下來。

排錯過程：怎麼找到真正的漏洞點

這是整題最花時間、也最有教學價值的部分——光是「猜」插入點在哪裡是不夠的，必須用除錯器實際追出真實呼叫鏈。

1. **TSTC** 查交易碼對應程式：**C002** → **SAPLCOK01** (跟 en04 用過的暖身呼叫 **CO_K01_GET_HEADER** 同一個 Function Group **COK01**)
2. **System→Status** 追蹤實際執行的程式：在 **C002** 開一張真實工單、點 Cost Analysis 之後，畫面顯示 **Program(GUI)=SAPLKKBC**——這是標準的 Report Painter/Report Writer 成本報表引擎 (跟交易碼 **KKBC_ORD** 是同一套)
3. ****ADT quickSearch K_KKB_*** 找候選進入點：找到 **K_KKB_CO_OBJECT_REPORT_CALL** (Function Group **KKB2**)，讀原始碼發現一個關鍵矛盾——函式裡真的寫了權限檢查 (**CALL FUNCTION 'G_JOB_AUTHORITY_CHECK' ... EXCEPTIONS NO_AUTHORITY = 1.**)，但這段檢查只存在於舊式 Report Painter 批次報表路徑；函式一開頭有個 **IF L_GRID = 'X'** 分支——這才是現代 ALV Grid 顯示路徑，呼叫 **K_KKB_CO_OBJECT_DISPLAY / K_KKB_KKBCS_ORDER_REPORT** 之後直接 **EXIT**，完全跳過**下面那段權限檢查
4. 在 **SE37** 對這支 **FM** 下中斷點，實際點 **CO02/CO03** 的 **Cost Analysis**，用除錯器逐步確認：
 - **L_GRID** 真的等於 **'X'**
 - 走的是 **l_global_object-aufnr** 有值的分支，呼叫 **K_KKB_KKBCS_ORDER_REPORT** (工單專屬報表)，緊接著就 **EXIT**。
 - 完整 Call Stack：**CO_COST_SHOW_ORDER...(SAPLCOCOST) → CK_F_CO_OBJECT_DISPLA...(SAPLKKCP) → CO_OBJECT_DISPLAY_WI...(SAPLCK10) → KKCK_CO_OBJECT_DISPL...(SAPLKKCK) → IF_CO_PC_CO_OBJECT~D...(Method) → K_KKB_CO_OBJECT_REPORT_CALL(SAPLKKB2)**

這一步證實了漏洞的真正成因：SAP 標準程式碼裡確實寫了權限檢查，但只覆蓋到舊路徑；現在實際在用的 ALV Grid 路徑是一塊真空地帶。這不是「SAP 忘記寫權限檢查」，而是新舊兩條路徑並存、新路徑漏補了同一段檢查——跟 en01 學到的「新舊技術並存、不是誰取代誰」是同一種系統演進現象，只是這次演進留下了一個安全缺口。

為什麼不能直接插在函式中間

en04 已經學過：Implicit Enhancement Point 只存在於 **FORM / FUNCTION / METHOD / MODULE** 的開頭或結尾，不能插在函式中間 (像 **IF L_GRID = 'X'** 分支裡面)。所以正確的插入點是 **K_KKB_CO_OBJECT_REPORT_CALL** 函式的最開頭——這樣不管走 Grid 還是舊路徑都能一併攔到，設計也更乾淨。

設計與建置

權限物件：標準 **K_ORDER** 欄位是 **RESPAREA / AUFART / AUTHPHASE / CO_ACTION / KSTAR**，沒有 **WERKS**，用不上，改自建 Z 權限物件：

- **SU21** 建立 **Z_EN08_COS** (10 碼上限，Object Class **CO**)，欄位 **ACTVT + WERKS** (皆為既有標準欄位，不用新建)
- 額外要點「Permitted Activities」按鈕維護 **ACTVT** 允許的值 (**03 = Display**)
- **SU21** 完全沒有 **ADT** 路徑 (連 quickSearch 對已知存在的標準物件 **K_ORDER** 都回空結果，比 Search Help / T-code 更徹底)，只能 GUI 建立

Message Class：**ZEN08**，訊息 **001**

- 空殼可以用 `ADT POST /sap/bc/adt/messageclass` 建立且真的生效
- ⚠️ 但訊息文字寫入 **ADT** 完全失敗：正確的 XML schema 是 `mc:message` 直接當 `mc:messageClass` 的子元素（不包一層 `mc:messages`），屬性 `mc:msgno / mc:msgtext`；PUT 回 200 OK，但讀回來還是空的，連補一次 Activation 都救不回來——這是本課程第二次遇到「回應成功但資料沒有真的落地」（第一次是 en01 ~ en07 課程外，Table Type 用錯 Content-Type 的案例），這次確認 schema / Content-Type 都對，判斷是這個環境的真實功能缺陷
- 最終文字只能請使用者到 SE91 手動補：`Plant &1 has no authorization for cost analysis`（後來依使用者建議改成更自然的措辭：「沒有權限顯示廠別 &1 的成本分析」——原本的英文措辭把「沒有權限」講成廠別的屬性，容易讓人誤讀成廠別本身沒有權限，而不是使用者沒有權限）

插入點：`K_KKB_CO_OBJECT_REPORT_CALL` 函式最開頭（SE37 → Edit → Enhancement Operations → Show Implicit Enhancement Options → Start of Function → Create Implementation，跟 en04 手法一致）

踩坑：**KOPF-WERKS** 抓不到值：第一版程式碼直接用函式的匯入參數 `KOPF LIKE CKI64A OPTIONAL` 裡的 **KOPF-WERKS** 判斷廠別，實測發現訊息跳出來時廠別是空白（`Plant has no authorization...`）。用除錯器檢查發現 **KOPF-WERKS** 真的是空的——原因是這個欄位只有在「用 **KOKRS** 反推廠別」那個分支才會被用到，**工單型的呼叫路徑（AUFNR 有值）** 完全不會去碰它，**CKI64A** 這個結構本質上是給「成本估算畫面」用的輔助欄位，不是工單專屬結構。

修正：動態 **ASSIGN** 讀取跨程式全域變數：改用 en04 第 23 節學過的技巧，直接指向 **SAPLCOK01** 全域的 **CAUFVD**（工單抬頭工作區）：

```
FIELD-SYMBOLS: <fs_werks> TYPE any.
DATA: lv_werks TYPE werks_d.

ASSIGN ('(SAPLCOK01)CAUFVD-WERKS') TO <fs_werks>.
IF <fs_werks> IS ASSIGNED.
    lv_werks = <fs_werks>.
ENDIF.

AUTHORITY-CHECK OBJECT 'Z_EN08_COS'
    ID 'ACTVT' FIELD '03'
    ID 'WERKS' FIELD lv_werks.
IF sy-subrc <> 0.
    MESSAGE e001(zen08) WITH lv_werks.
ENDIF.
```

因為這次是真的從 **CO02/CO03** 交易一路呼叫下來（不像 en04 的獨立測試程式需要先做一次「暖身呼叫」讓目標程式載入記憶體），**SAPLCOK01** 這時候必然已經在記憶體裡，所以不需要暖身呼叫，**ASSIGN** 直接就成功了。

驗證結果

使用者親自用真實工單（**Plant 1011**）在 **CO02/CO03** 測試兩種情境，完整端對端驗證成功：

- 有 **Z_EN08_COS** 權限：正常進入 Cost Analysis 報表（Target/Actual Comparison，顯示真實成本資料）

- 沒有 **Z_EN08_COS** 權限：畫面狀態列跳出「沒有權限顯示廠別 1011 的成本分析」，畫面被正確擋下，無法看到成本資料

案例一：PPCO0001 (工單存檔時 Component Change 存 Log Table) ——待建置

需求：工單存檔時，把使用者對工單元件 (Component) 的異動內容 (改前/改後值) 記錄進自訂 Log 表，方便事後稽核。

已查證的技術背景：

- Enhancement 名稱 **PPCO0001**，Function Exit 是 **EXIT_SAPLCOBT_001** (Function Group **SAPLCOBT**)
- 查 **MODACT** 表：本系統 0 筆，從未被任何 **CMOD Project** 指派過，是乾淨的教學素材
- 使用者提供完整的真實參考文件 (工單存檔時將 **Component Change** 存入 **Log Table.docx**)，內含實際運作過的實作邏輯：核心是 **FORM log_co02_change_content** (放在 Include **ZXC01F01**)，逐欄位比對元件變更前後的值、寫入自訂 Log 表 (範例用 **ZPPT005**)，過程含 **ENQUEUE/DEQUEUE** 鎖定與使用者/時間戳記錄

下一步：比照 en02 的 CMOD 四步驟 (Project 建立 → Assign **SAPLCOBT** → 雙擊生成 Include → Activate)，設計本課程自己的 Log 表與 Include 內容 (沿用參考文件的邏輯架構，物件命名改用課程慣例)。

案例三：MB52 加 MRP Controller 選取欄位 (Explicit Enhancement，消費者視角) ——待建置

需求：**MB52** (顯示倉庫庫存) 的選取畫面，除了既有的 Purchasing Group 之外，再加一個 MRP Controller (**MARC-DISPO**) 選取欄位。

已查證的技術背景：

- **MB52** 底層程式是 **RM07MLBS**
- 這支標準程式已經存在一個真實的 Explicit Enhancement Point：**ENHANCEMENT-POINT RM07MLBS_3 SPOTS ES_RM07MLBS STATIC**，上面已經掛了別人 (**MGV_LAMA_RM07MLBS**) 做的 Implementation，加了一個 **mfrpn** (**PIC-Supersession/MPN**) 欄位——這證實這個插入點是真實生效、可以再掛新 Implementation 的
- 使用者提供真實參考文件 (**MB52 加MRP Controller.docx**)，內含實際操作截圖：在同一個 Enhancement Point 上用 **Create Implementation** 新建了 **ZMB52_ADD_MRPCONTROLLER**，加上：

```
``abap SELECT-OPTIONS: s_dispo FOR marc-dispo. ` 並用 SELECTION_TEXTS_MODIFY 動態設定這個新欄位的選取畫面文字 ( 因為新增的欄位沒辦法用標準 Text Elements 維護 )：`abap DATA: lt_seltexts LIKE rsseltexts OCCURS 0 WITH HEADER LINE. lt_seltexts-name = 'DISPO'. lt_seltexts-kind = 'S'. lt_seltexts-text = 'Mrp controller'. APPEND lt_seltexts. CALL FUNCTION 'SELECTION_TEXTS_MODIFY' EXPORTING PROGRAM = 'RM07MLBS' TABLES SELTEXTS = lt_seltexts. ``
```

跟 en07 的關鍵差異：en07 是「生產者視角」，Enhancement Spot (**ENHANCEMENT-POINT/ENHANCEMENT-SECTION** 那一行本身) 是自己宣告的；這裡是「消費者視角」——**ENHANCEMENT-POINT RM07MLBS_3** 這一行已經存在於標準程式裡 (別人留的)，這次要做的只有 **Create Implementation** 這一步，不需要 (也不能) 碰 **ENHANCEMENT-POINT** 宣告本身。

下一步：SE38 在 RM07MLBS 找到 RM07MLBS_3 這個插入點，比照參考文件的做法新建 Implementation，加上 DISPO 選取欄位，實際在 MB52 測試選取畫面是否正確出現新欄位、篩選邏輯是否生效。

學習目標

- 能講出「生產者視角」（en07：自己宣告插入點）跟「消費者視角」（en08：找 SAP 已留好的插入點）的差異，並知道真實工作裡消費者視角才是常態
- 能示範完整的「消費者視角」排錯流程：TSTC 查程式 → System→Status 追實際執行路徑 → quickSearch 找候選 FM → 讀原始碼判斷邏輯缺口 → 除錯器下中斷點實際驗證
- 能解釋為什麼「標準程式裡寫了權限檢查」不代表「所有執行路徑都會走到這段檢查」——新舊路徑並存時，舊路徑的檢查不會自動套用到新路徑
- 知道 Implicit Enhancement Point 只能插在 FORM/FUNCTION/METHOD/MODULE 的開頭或結尾，設計插入點位置時要往「函式最外層」找，不能想插在流程中間
- 能講出動態 ASSIGN 讀取跨程式全域變數的技巧，並判斷「當下情境是否需要暖身呼叫」（真實交易鏈觸發 vs 獨立測試程式）
- 知道 SU21（權限物件）完全沒有 ADT 路徑，是本課程遇過最徹底的 GUI-only 限制之一
- 知道 Message Class 的空殼可以用 ADT 建立，但訊息文字目前這個環境無法透過 ADT 寫入，這是「回應成功但資料沒有真的落地」的又一個真實案例
- 能講出「消費者視角找到既有 Explicit Enhancement Point」跟「生產者視角自己宣告」在操作上的差異：前者只需要 Create Implementation，不用碰宣告本身

事前準備（已於本系統 client 130 實際完成，非假設）

1. Z_EN08_COS（權限物件，使用者於 SU21 建立）：Object Class CO，欄位 ACTVT+WERKS，Permitted Activities 已維護 03。
2. ZEN08（Message Class，ADT 建立空殼成功；訊息 001 文字使用者於 SE91 補上）：「沒有權限顯示廠別 &1 的成本分析」。
3. ZIM_CO_COST_AUTH（Implicit Enhancement Implementation，使用者於 SE37 建立，掛在 K_KKB_CO_OBJECT_REPORT_CALL 函式最開頭）：內容見上方案例二程式碼，已使用者親自用 CO02/CO03 真實工單（Plant 1011）雙重情境（有權限 / 無權限）端對端驗證成功。
4. 案例一（PPCO0001）、案例三（MB52）尚未建置，僅完成需求分析與技術背景查證（含使用者提供的真實參考文件內容）。

題目需求

1. 畫出案例二的完整排錯流程圖：從「使用者反映 Cost Analysis 不卡權限」到「找到 K_KKB_CO_OBJECT_REPORT_CALL 函式開頭是正確插入點」，中間經過哪些工具 / 步驟？每一步驗證了什麼、排除了什麼可能性？
2. 解釋為什麼 G_JOB_AUTHORITY_CHECK 這段程式碼「存在」不代表「權限漏洞不存在」：這個現象背後反映了什麼樣的系統演進模式？你會怎麼跟不熟 ABAP 的專案經理解釋這個矛盾？
3. 解釋為什麼要先用 KOPF-WERKS 失敗、才改用 ASSIGN（'（SAPLCOK01）CAUFVD-WERKS'）：這兩個資料來源的本質差異是什麼？如果一開始就去查 CKI64A 這個結構的用途說明，有沒有機會提早避開這個坑？
4. 比較案例二（Implicit）跟案例三（Explicit，消費者視角）的操作步驟差異：兩者都是要在標準程式裡「加東西」，為什麼案例二需要在 SE37 走「Show Implicit Enhancement Options」，案例三卻是直接對已存在的 ENHANCEMENT-POINT 做 Create Implementation？

參考答案

案例二完整排錯流程圖：

- ① TSTC 查 C002 對應程式
 - 確認底層是 SAPLCOK01 (跟 en04 同一個 Function Group COK01)
 - ↓
- ② 實際點 Cost Analysis · System→Status 看 Program(GUI)
 - 確認真正在跑的是 SAPLKKBC (Report Painter/KKBC_ORD 引擎) , 排除了「Cost Analysis 走別的完全無關程式」的可能性
 - ↓
- ③ ADT quickSearch K_KKB_* 找候選 FM · 讀原始碼
 - 找到 K_KKB_CO_OBJECT_REPORT_CALL , 發現「有權限檢查、但只在舊路徑」的矛盾, 排除了「SAP 完全沒寫過權限檢查」的假設
 - ↓
- ④ SE37 對該 FM 下中斷點, 實際操作驗證
 - 確認 L_GRID='X'、確認真的走到 K_KKB_KKBCS_ORDER_REPORT 分支、確認緊接著 EXIT.、確認 Call Stack 完整路徑
 - 這一步排除了「①~③的推論是錯的」這個風險, 是唯一的「真正執行證據」, 前面三步都只是縮小懷疑範圍

「權限檢查存在≠沒有漏洞」的系統演進模式：SAP 標準程式碼往往不是一次寫成的，Report Painter 這套成本報表引擎有新舊兩種顯示路徑（傳統批次報表 vs 現代 ALV Grid），推測是 ALV Grid 路徑是後來才加上去的效能優化/使用者體驗改進，加的時候複製了大部分邏輯，但沒有連帶把權限檢查那段也複製過去——這在大型遺留系統的維護裡很常見：新舊功能路徑並存，新路徑補完既有邏輯時容易漏掉「不顯眼但重要」的部分（權限檢查通常不影響功能是否「看起來正常」，很容易被測試忽略）。跟專案經理解釋可以這樣說：「這不是 SAP 沒做過權限檢查，是系統升級時新增了一條更快的路，這條新路沒有把舊路上的安全閘一起搬過去，兩條路平常看起來結果一樣，只有『誰能看什麼』這件事不一樣」。

KOPF-WERKS vs CAUFVD-WERKS 的本質差異：KOPF (型別 CKI64A) 是「顯示成本估算畫面的輔助欄位」，語意上服務的是物料成本估算 (Costing) 情境，不是「工單」情境——即使它剛好有 WERKS 欄位，也只在特定分支 (用 KOKRS 反推廠別) 才會被填值，工單導向的呼叫路徑完全不會經過那段邏輯。CAUFVD 才是真正的「工單抬頭」全域工作區，任何工單相關畫面在載入時都會把它填好。如果一開始就用 SE11 或 ADT 查一下 CKI64A 的 Short Description (「Auxiliary Fields on Screens for Displaying Cost Estimates」) ，就會看到它跟「工單」完全沒有直接關聯，能提早懷疑這個欄位在工單情境下不可靠——這是一個很好的提醒：拿到一個結構要用之前，先看一眼它的用途說明，不要只看「剛好有我要的欄位名稱」就直接用。

案例二 vs 案例三操作步驟差異：案例二要插入的位置 (K_KKB_CO_OBJECT_REPORT_CALL 函式最開頭) 是一個系統自動保證存在、但預設隱藏的 Implicit 插入點——任何 FORM/FUNCTION/METHOD 的頭尾都有，只是要切換「Show Implicit Enhancement Options」才看得到，因為它「本來就在那裡」，不需要任何人事先宣告。案例三要用的 ENHANCEMENT-POINT RM07MLBS_3 則是一個已經被某個開發者明確宣告過的 Explicit 插入點 (en07 教過這一步需要 Create Option) ，這個宣告動作已經有人做過了 (做出 mfrpn 那個 Implementation 的人) ，現在只是要在同一個位置加另一個 Implementation，所以只需要 Create Implementation，完全不用碰 Create Option 那一步——這正是 en07 (生產者視角，自己做 Create Option) 跟 en08 案例三 (消費者視角，別人已經做過 Create Option) 的操作步驟差異根源。

思考題

1. 案例二的排錯過程用了四個步驟 (TSTC / System→Status / quickSearch / 除錯器)，如果只做到第三步 (讀原始碼判斷) 就直接動手寫 Enhancement，沒有做第四步的除錯器驗證，可能會踩到什麼風險？ (提示：原始碼裡的分支邏輯可能有好幾層 IF，光用眼睛讀不一定能百分之百確定「這次呼叫真的會走到我以為的那個分支」)
2. SU21 完全沒有 ADT 路徑、Message Class 訊息文字寫入 ADT 也失敗——這兩個限制加在一起，代表案例二這整個功能要正式上線，有哪些步驟是「Claude 完全幫不上忙，一定要人工在 GUI 完成」的？如果團隊要把這類「AI 協作但有 GUI-only 缺口」的工作流程寫成標準作業程序，你會怎麼分配「Claude 做什麼、人做什麼」？
3. 案例三預計要在標準程式 RM07MLBS 上掛新的 Implementation，這是一個很多人日常在用的標準交易 (MB52)。動手之前，你會做哪些「安全閘」設計，確保新增的 DISPO 選取欄位不會影響到還沒設定這個欄位的既有查詢行為？ (提示：想想 en04 / en06 用過的安全閘模式——只在特定條件下才生效——這裡的「特定條件」該怎麼定義，才能保證「使用者沒填 DISPO」時行為跟改之前完全一樣)